

Product Requirements Document: Kanji Scanner (Working Title)

Version: 1.0 (Prototype / MVP) **Author:** あさ (Product Manager & Technical Architect) **Status:** Draft for Development **Date:** August 21, 2026

1. Executive Summary & Goals

Kanji Scanner is a prototype iOS application that lets native English-speaking Japanese learners point their camera at printed Japanese text and instantly see the reading and meaning of any kanji or kanji compound/word in frame — fully offline. The core design principle is **frictionless simplicity**: no accounts, no learning-app clutter, no setup. Point, capture, read.

The product is built for a real, recurring moment: a learner encounters a road sign they can't read while exploring Osaka, or is reading a book on the subway or on an airplane with no cellular signal, and hits an unfamiliar kanji or word. Existing Japanese OCR/dictionary apps solve this but bury it under flashcard systems, SRS schedules, and account walls. Kanji Scanner strips all of that away for v1 and does one thing well.

Goals for v1

1. Ship a functioning, fully offline, on-device OCR + dictionary lookup tool for printed Japanese text.
2. Support both isolated kanji and kanji compounds/words, with results displayed hierarchically (word first, constituent kanji below), similar to the [Yomitan browser extension's](#) pop-up dictionary pattern.
3. Validate the core hypothesis — that a stripped-down, single-purpose scanning tool has product-market fit — before investing in cloud OCR, accounts, or learning mechanics.
4. Keep the codebase and scope manageable for a solo developer to build, ship, and maintain.

Non-Goals for v1

- Building a comprehensive Japanese learning platform
 - Supporting handwritten text
 - Requiring or offering user accounts
 - Cloud-dependent recognition
-

2. Problem Statement

Native English-speaking learners of Japanese constantly encounter kanji "in the wild" — on subway signage, restaurant menus, product packaging, and in books — that they don't yet recognize. Looking up an unknown kanji today typically requires:

- Manually drawing the character stroke-by-stroke in a dictionary app (slow, error-prone if you don't know the correct stroke order)
- Searching by radical (requires already knowing kanji decomposition)
- Typing a guessed romaji reading, which is impossible if you don't know the reading. Even when you know the kanji, there can be multiple onyomi and kunyomis, leaving the reader to at best make an informed guess. With the many exceptions in Japanese readings, mistakes are bound to happen.

All of these methods are extremely time consuming - this friction is highest in exactly the moments learners most need a quick answer: mid-commute, mid-conversation, or mid-page, often with poor or no connectivity (subway tunnels, airplanes, international roaming). The delay or difficulty of lookup breaks reading flow and discourages learners from engaging with authentic, real-world Japanese material outside of curated textbooks and apps.

Existing camera-based OCR dictionary apps address part of this problem, but tend to bundle in SRS flashcard systems, level-tracking, account creation, and cluttered UI aimed at retention/engagement metrics — adding friction and cognitive load to what should be a two-second lookup.

The gap: there is no dead-simple, fully offline, camera-first kanji lookup tool built for the single moment of "I see a character I don't know — tell me what it means, right now."

3. User Personas & User Stories

Persona 1: The Commuter Learner (Primary)

Sam is an intermediate Japanese learner living in New York City. Sam reads Japanese novels and manga during subway commutes, where connectivity is unreliable or nonexistent between stations. Sam knows enough grammar to read simple sentences but regularly hits unfamiliar kanji and compounds. Sam wants an answer in seconds, without breaking reading flow, and without the phone needing a signal.

Persona 2: The Traveler / Casual Encounterer (Secondary)

Jordan is visiting Japan and encounters kanji on menus, signs, and packaging. Jordan has basic-to-intermediate Japanese knowledge and wants quick, occasional lookups — not a study session. Jordan may also have limited or expensive international data access, making offline function valuable.

User Stories & Acceptance Criteria

US-1: Capture a photo of printed Japanese text As a learner, I want to open the app and immediately point my camera at text, so that I can capture kanji with minimal steps.

Acceptance Criteria:

- App opens directly to a live camera viewfinder (no landing page, no login)
- User can tap a capture button to take a still photo, or the app can be configured to attempt live/continuous recognition (stretch goal — see Section 4)
- Camera permission is requested on first launch with a clear explanation of use
- If camera permission is denied, the app shows a clear message directing the user to Settings

US-2: Select the specific text region to look up As a learner, I want to select the specific kanji/word I'm interested in, so that I get an accurate, targeted lookup even if other text is in frame.

Acceptance Criteria:

- After capture, the app overlays bounding boxes around all detected text regions
- No cropping - this adds additional clicks for a user. The user should be able to simply tap a specific detected word/kanji region to select it for lookup
- If only one clear text region is detected, the app may auto-select it to save a step
- User can retake the photo if no usable text was detected

US-3: View dictionary results for an isolated kanji As a learner, I want to see the reading(s) and meaning(s) of a single kanji I've scanned, so that I understand what it means and how to say it.

Acceptance Criteria:

- Results screen displays: the kanji character, on'yomi reading(s) in katakana, kun'yomi reading(s) in hiragana, and English meaning(s)

- If multiple meanings exist, all are shown, ranked by common usage where data supports it
- Results render within 2 seconds of selection on a modern iPhone (offline, on-device)

US-4: View dictionary results for a kanji compound/word As a learner, I want to see the compound word's reading and meaning first, with each constituent kanji broken out below it, so that I understand both the word as a whole and its building blocks — similar to Yomitan's pop-up dictionary format.

Acceptance Criteria:

- If the selected/detected text is a recognized multi-kanji compound, the results screen shows the compound word, its reading (furigana/kana), and its meaning(s) at the top
- Below the compound entry, each individual kanji in the compound is listed separately with its own reading(s) and meaning(s)
- If the compound is not found in the dictionary but individual kanji are recognized, the app gracefully falls back to showing kanji-level results only, with a clear indicator that no compound match was found

US-5: Use the app with no network connection As a learner, I want the app to work exactly the same on a subway with no signal or in airplane mode as it does with full connectivity, so that I never lose functionality when I need it most.

Acceptance Criteria:

- OCR recognition, kanji/word segmentation, and dictionary lookup all function fully offline
- No feature in v1 requires a network call
- App does not display network error states for core scanning/lookup functionality, since none is needed

US-6: Handle unrecognized or low-confidence scans gracefully As a learner, I want clear feedback when the app can't confidently identify text, so that I know to retake the photo or try a tighter crop rather than getting a wrong answer.

Acceptance Criteria:

- If OCR confidence falls below a defined threshold, the app displays a "couldn't confidently read this — try retaking the photo" message rather than a guessed result
- If text is recognized but not found in the dictionary (e.g., a proper noun, non-standard character), the app clearly states "no dictionary entry found" rather than showing a blank or broken screen

US-7: Retain the option to re-scan quickly As a learner, I want to return to the camera quickly after viewing a result, so that I can look up the next unfamiliar kanji without extra taps.

Acceptance Criteria:

- A single, obvious action (e.g., a persistent "scan again" button or swipe gesture) returns the user from the results screen to the live camera view
 - Returning to camera does not require re-granting permissions or reloading the app
-

4. Core Functional Requirements

4.1 Image Capture

- FR-1: The app shall provide a live camera viewfinder as the default/home screen.
- FR-2: The app shall allow the user to capture a still image via a tap-to-capture control.
- FR-3 (Stretch, not required for v1 ship): The app may support near-real-time recognition overlay without requiring an explicit capture tap, similar to Google Translate's live camera mode. This is explicitly a stretch goal and is descope for the mvp.

4.2 Text Recognition (OCR)

- FR-4: The app shall use Apple's Vision framework (`VNRecognizeTextRequest`, Japanese language configuration) to perform on-device text recognition on the captured image.
- FR-5: The app shall detect and return bounding boxes for distinct text regions/lines within the captured image.
- FR-6: The app shall apply a minimum confidence threshold to OCR results; results below threshold shall trigger the graceful-failure state defined in US-6.
- FR-7: Recognition shall function with zero network connectivity, in Airplane Mode, and with cellular/Wi-Fi fully disabled.

4.3 Text Segmentation (Kanji vs. Compound Detection)

- FR-8: After OCR returns raw text, the app shall segment recognized Japanese text into candidate lookup units — both individual kanji and multi-character compounds — prior to dictionary lookup.
- FR-9: Segmentation shall prioritize longest-match compound lookups first (i.e., attempt to match the longest valid dictionary word starting at the selected point), falling back to single-kanji lookup if no compound match is found. This mirrors standard J-E dictionary tokenization behavior (as used by tools like Yomitan/Mecab-based segmenters).

4.4 Dictionary Lookup

- FR-10: The app shall bundle a local, offline dictionary database derived from KANJIDIC2 (kanji-level data: readings, meanings, stroke count, grade, frequency) and JMdict (word/compound-level data: readings, meanings, part of speech), both sourced from the Electronic Dictionary Research and Development Group (EDRDG) under their Creative Commons Attribution-ShareAlike license.
- FR-11: The app shall query this dataset via a local SQLite database (converted from source XML at build time) for fast, indexed, offline lookups.
- FR-12: For compound/word queries, results shall return the compound entry plus references to each constituent kanji's individual entry, enabling the hierarchical display defined in US-4.
- FR-13: The app's data schema shall store JLPT level, WaniKani-style level, and common-use/frequency flags where available in source data, even though these fields will not be surfaced in the v1 UI. This avoids a schema migration when tagging is added in a later version.

4.5 Results Display

- FR-14: The results screen shall display, for a compound: the compound word, its reading, its meaning(s), followed by each constituent kanji with its own readings and meanings — visually distinguishing the "word" entry from the "kanji breakdown" entries (e.g., via typography/section hierarchy), matching the Yomitan-style pattern described by the user.
- FR-15: The results screen shall display, for an isolated kanji: the character, on'yomi, kun'yomi, and meaning(s).
- FR-16: The UI shall prioritize legibility and simplicity — large character display, minimal chrome, no more than one primary action visible at a time (e.g., "scan again").

4.6 Performance

- FR-17: End-to-end latency from photo capture to results display shall not exceed 2 seconds on an iPhone 13 or newer, under normal lighting conditions, fully offline.
- FR-18: The app shall not require any loading spinner longer than 2 seconds without user-facing progress feedback.

4.7 Explicitly Not Included in v1 (see also Section 6)

- No search history persistence (planned for a near-term future release)
 - No manual text search/typed lookup — camera capture is the only input method for v1
 - No settings/preferences screen beyond OS-level permissions
-

5. Data Models / Database Schema

The app will ship with a pre-built, read-only SQLite database bundled in the app package, generated at build time from KANJIDIC2 and JMdict XML sources. The schema below anticipates future features (history, tagging) without requiring migration.

5.1 `kanji` table

Stores individual kanji character data (sourced from KANJIDIC2).

Column	Type	Notes
<code>id</code>	INTEGER (PK)	Internal identifier
<code>character</code>	TEXT	The kanji character itself (unique)
<code>onyomi</code>	TEXT (JSON array)	On'yomi readings, katakana
<code>kunyomi</code>	TEXT (JSON array)	Kun'yomi readings, hiragana
<code>meanings</code>	TEXT (JSON array)	English meanings, ordered by relevance
<code>stroke_count</code>	INTEGER	For future stroke-order features
<code>grade</code>	INTEGER, nullable	Japanese school grade level, if available
<code>jlpt_level</code>	INTEGER, nullable	N5–N1; not surfaced in v1 UI, reserved for future tagging
<code>frequency_rank</code>	INTEGER, nullable	Newspaper frequency ranking, if available
<code>radical</code>	TEXT, nullable	Primary radical, reserved for future search-by-radical feature

5.2 `words` table

Stores compound/word-level entries (sourced from JMdict).

Column	Type	Notes
<code>id</code>	INTEGER (PK)	Internal identifier
<code>surface_form</code>	TEXT	The word as written in kanji/kana (e.g., 漢字)
<code>reading</code>	TEXT	Full reading in kana (e.g., かんじ)
<code>meanings</code>	TEXT (JSON array)	English meaning(s), ordered by sense/relevance
<code>part_of_speech</code>	TEXT, nullable	e.g., noun, verb — reserved, not required for v1 UI
<code>is_common</code>	BOOLEAN	Flag from JMdict "common word" marker; reserved for future frequency tagging

5.3 `word_kanji_map` table (junction table)

Maps each word to its constituent kanji, enabling the hierarchical "word → kanji breakdown" display.

Column	Type	Notes
<code>word_id</code>	INTEGER (FK → words.id)	
<code>kanji_id</code>	INTEGER (FK → kanji.id)	
<code>position</code>	INTEGER	Character position within the word, for ordered display

5.4 Indexing

- Index on `kanji.character` for $O(\log n)$ direct lookup
- Index on `words.surface_form` for compound lookup during longest-match segmentation
- Composite index on `word_kanji_map` (`word_id`, `position`) for fast, ordered kanji breakdown retrieval

5.5 Deferred (not built in v1, schema-compatible for later addition)

- **scan_history** table (timestamp, matched word/kanji ID, thumbnail reference) — planned for a near-future release per user's stated roadmap; no persistence logic will be built in v1, but the above schema does not preclude adding this table without migrating existing data.
-

6. Out of Scope (v1)

The following are explicitly excluded from the v1 prototype to protect scope and timeline for a solo developer:

- **Cloud or hybrid OCR** — v1 is on-device (Apple Vision framework) only. Cloud/hybrid may be evaluated in a future version based on MVP accuracy testing.
 - **Handwritten text recognition** — v1 supports printed text only.
 - **Full sentence parsing / machine translation** — v1 performs kanji and compound-word lookup only, not sentence-level grammar parsing or translation.
 - **User accounts and cloud sync** — all data is local to device; no login, no backend.
 - **Search history** — planned for a near-term future release, not built in v1 (schema will not block adding it later).
 - **Manual/typed search** — camera capture is the sole input method in v1; no keyboard-based dictionary search.
 - **Learning mechanics** — no flashcards, spaced repetition (SRS), quizzes, or progress tracking of any kind.
 - **JLPT / WaniKani / frequency tag display in UI** — data fields exist in the schema for future use but are not shown to the user in v1.
 - **iPad or multi-device support** — iPhone only for v1.
 - **Android or other platforms** — iOS only.
 - **Non-Japanese languages** — Japanese-to-English only; no support for other language pairs.
 - **Live/continuous camera recognition** — treated as a stretch goal only (FR-3); not a v1 commitment.
-

7. Success Metrics (KPIs)

Given this is a solo-developer prototype without a marketing or growth budget, v1 success metrics are intentionally scoped around **functional validation and personal/informal real-world usability**, rather than large-scale product analytics.

Recognition & Lookup Quality

- **OCR success rate:** ≥90% of clearly printed, in-focus kanji/compounds are correctly recognized by the Vision framework in real-world test conditions (books, signage, packaging).
- **Dictionary match rate:** ≥95% of correctly OCR'd kanji/compounds return a valid dictionary entry (accounting for the fact that some recognized text, like proper nouns, may legitimately have no entry).
- **Compound-first display accuracy:** ≥90% of multi-kanji scans correctly identify and prioritize the longest valid compound match before falling back to individual kanji.

Performance

- **End-to-end latency:** ≥95% of scans return results within the 2-second target defined in FR-17, on iPhone 13 or newer.
- **Offline reliability:** 100% of core scan-to-result functionality works with zero connectivity (validated via Airplane Mode testing).

Usability (Informal / Self-Testing for Prototype Stage)

- **Time-to-result:** From app open to viewing a result, the full flow should take no more than ~5 seconds for a motivated first-time user, validated via informal self-testing (e.g., real subway/reading sessions).
- **Retake rate:** Track (informally, via manual use) how often a scan requires a retake or recrop due to low-confidence OCR — target trending downward as capture UX is refined.

Technical Health

- **Crash-free session rate:** ≥99% across manual testing sessions.
- **App bundle size:** Keep the compiled SQLite dictionary + app under a reasonable size for a lightweight prototype (target: under 150MB total app size) — monitored as the JMdict/KANJIDIC2 import pipeline is built.

Qualitative Validation (Prototype-Stage Proxy for Product-Market Fit)

- Since v1 has no accounts/analytics backend, the primary "success" signal at this stage is direct, repeated personal use during real commuting/reading scenarios — i.e., does the builder (and any early testers) reach for this app instead of an existing alternative when they hit an unknown kanji in the wild. This qualitative signal should directly inform whether to invest in cloud/hybrid OCR, search history, or tagging features in v1.1.
-

Appendix: Key Technical Decisions Summary

Decision	Choice	Rationale
OCR approach	On-device, Apple Vision framework	Offline-first requirement, zero API cost, sufficient accuracy for printed text
Recognition scope	Single kanji + compounds	Matches real-world reading use case; compound-first display mirrors Yomitan UX
Source material	Printed text only	Handwriting adds significant model complexity; deferred to future roadmap
Dictionary data	Bundled offline (KANJI DIC2 + JMdict via SQLite)	Only approach that satisfies offline requirement; same data quality as live API
Learning features	None in v1	Deliberate differentiation from "bloated" competitor apps; simplicity is the product
Accounts/persistence	Local device storage only	No backend needed for a lookup-only tool
Min iOS version	iOS 17.0	Modern Swift/SwiftUI tooling reduces solo-dev complexity; high adoption by 2026
Device support	iPhone only	Solo developer, prototype scope